Big data analytics workflows with Databricks

Lecture 3 — Big data file formats & Delta Lake

Andrii Muzychuk

UCU, 2026

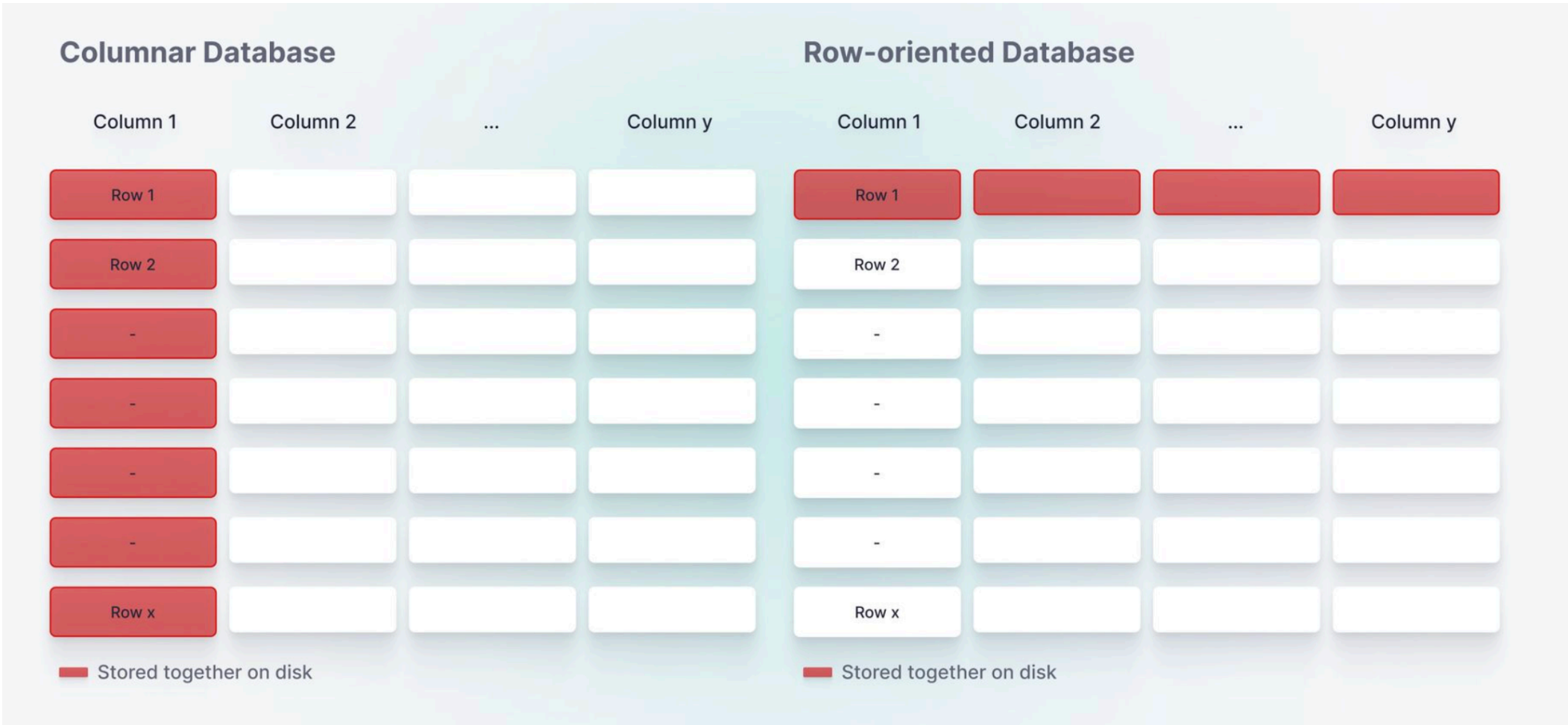
POWERED BY APPLIED SCIENCES FACULTY

Lecture 3 — Big data file formats & Delta Lake

Overview

- Storage mechanisms
- Avro
- ORC
- Apache Parquet
- Delta Lake
- Any other alternatives?

Storage mechanisms



Learn more: What is a columnar database?

Example

Consider an example of `employee` table :

ID	Name	Age	Department
1	John	35	IT
2	Jane	28	HR
3	Bob	42	Finance

organized by record:

Row 1	1,John,35,IT
Row 2	2,Jane,28,HR
Row 3	3,Bob,42,Finance

stored column-wise:

1,2,3; John,Jane,Bob; 35,28,42; IT,HR,Finance

Row vs Columnar storage

- data is organized by record
- data associated with a specific record is stored adjacently
- examples: CSV, TSV, JSON, Avro.

- stores data column-wise to the computer disk.
- the values of each table column (field) are stored next to each other
- examples: Parquet, ORC, Delta, Hudi

Row vs. Columnar

	Row	Columnar
High performance of analytical queries	*	***
Efficient storage and compression	*	***
Easy to scale	*	***
Efficient for transactional workloads	***	*
Easy to implement and understand	***	*
Easy to modify specific record	***	*

Query example

Q: Take a look at the queries below. Which one would benefit from row and which from columnar formats?

Query 1:

```sql
SELECT mean(Age) AS avg_age
FROM employee
WHERE ID > 2
```

Query 2:

```sql
SELECT ID, Name, Age
FROM employee
WHERE Department = 'IT'
```

Learn more: The difference between row and columnar formats

CSV vs Parquet performance

- **CSV** - a simple, ubiquitous *row* format used by nearly every tool, and the de facto default for data processing pipelines
- **Parquet** - compressed, efficient *columnar* data representation available to any project in the Hadoop ecosystem.

Dataset	Size on Amazon S3	Query Run Time	Data Scanned	Cost
Data stored as CSV files	1 TB	236 seconds	1.15 TB	$5.75
Data stored in Apache Parquet Format	130 GB	6.78 seconds	2.51 GB	$0.01
Savings	87% less when using Parquet	34x faster	99% less data scanned	99.7% savings

Learn more: Difference Between Parquet and CSV

Common big data formats









Avro



- released late in 2009 as a **row-based**, language-neutral, schema-based serialization technique
- open-source format that is often recommended for Apache Kafka
- supports schema evolution
- write-heavy operations

Avro



- released late in 2009 as a **row-based**, language-neutral, schema-based serialization technique
- open-source format that is often recommended for Apache Kafka
- supports schema evolution
- write-heavy operations

```
{"namespace": "example.avro",
 "type": "record",
 "name": "User",
 "fields": [
     {"name": "name", "type": "string"},
     {"name": "favorite_number",  "type": ["int", "null"]},
     {"name": "favorite_color", "type": ["string", "null"]}
 ]
}
```

ORC (Optimized Row Columnar)



- ORC was introduced by Hortonworks in 2013 as **columnar** data format highly optimized to process data in Hive[1]
- when reads >> writes.
- when you rely on Hive and compression flexibility/options are key.
- supports schema evolution

Apache Parquet



- introduced in 2013 as a **columnar** storage format available to any project in the Hadoop ecosystem, regardless of the choice of data processing framework, data model or programming language.
- excels in write-once, read-many analytics scenarios
- offering highly efficient data compression and decompression
- supports data skipping
- offer multiple compression options and support schema evolution

Learn more: Parquet Documentation

Choosing file format: things to consider

- Read/Write Intensive & Query Pattern
- Compression
- Schema Evolution
- Nested Columns
- Platform Support

Choosing file format: things to consider

There is no silver bullet.



- write heavy operations: use row based format
- read heavy operations: use columnar format
- final choice depends on your goals, use case, amount of data and budget

Challenging tasks for parquet

- updates / deletes
- ACID transactions guaranties
- data versioning
- dealing with many small files

Delta lake



Delta lake



- open source project that **enables a Lakehouse architecture** on top of data lakes.

Delta lake



- open source project that **enables a Lakehouse architecture** on top of data lakes.
- technically Delta lake is an abstraction layer on top of the parquet

Delta lake



- open source project that **enables a Lakehouse architecture** on top of data lakes.
- technically Delta lake is an abstraction layer on top of the parquet
- **ACID transactions** on Spark: ensure that readers never see inconsistent data

Delta lake



- open source project that **enables a Lakehouse architecture** on top of data lakes.
- technically Delta lake is an abstraction layer on top of the parquet
- **ACID transactions** on Spark: ensure that readers never see inconsistent data
- **Streaming and batch unification**: A table in Delta Lake is a batch table as well as a streaming source and sink

Delta lake



- open source project that **enables a Lakehouse architecture** on top of data lakes.
- technically Delta lake is an abstraction layer on top of the parquet
- **ACID transactions** on Spark: ensure that readers never see inconsistent data
- **Streaming and batch unification**: A table in Delta Lake is a batch table as well as a streaming source and sink
- **Time travel**: Data versioning enables rollbacks, full historical audit trails, and reproducible machine learning experiments

Delta lake



- open source project that **enables a Lakehouse architecture** on top of data lakes.
- technically Delta lake is an abstraction layer on top of the parquet
- **ACID transactions** on Spark: ensure that readers never see inconsistent data
- **Streaming and batch unification**: A table in Delta Lake is a batch table as well as a streaming source and sink
- **Time travel**: Data versioning enables rollbacks, full historical audit trails, and reproducible machine learning experiments
- **Upserts and deletes**: Supports merge, update and delete operations to enable complex use cases like change-data-capture, slowly-changing-dimension (SCD) operations, streaming upserts, etc.

Delta lake



- open source project that **enables a Lakehouse architecture** on top of data lakes.
- technically Delta lake is an abstraction layer on top of the parquet
- **ACID transactions** on Spark: ensure that readers never see inconsistent data
- **Streaming and batch unification**: A table in Delta Lake is a batch table as well as a streaming source and sink
- **Time travel**: Data versioning enables rollbacks, full historical audit trails, and reproducible machine learning experiments
- **Upserts and deletes**: Supports merge, update and delete operations to enable complex use cases like change-data-capture, slowly-changing-dimension (SCD) operations, streaming upserts, etc.

Learn more: Delta Lake Documentation

Is Delta the only option?

No, there are two alternatives Apache Iceberg and Apache Hudi.

Analyzing and Comparing Lakehouse Storage Systems

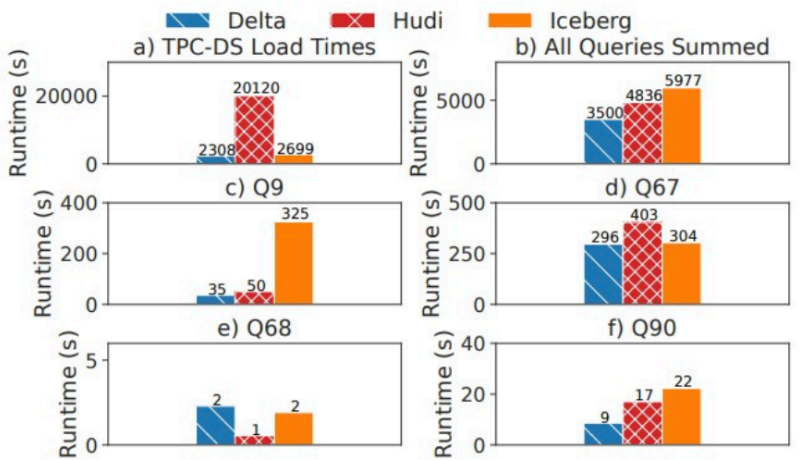


Figure 1: Comparison of Delta Lake, Hudi and Iceberg 3 TB TPC-DS load and query times, including four queries with large differences.

Learn more:

- Paper: Comparing Lakehouse Storage Systems
- Apache Iceberg vs Delta Lake: Features, Differences & Use Cases
- Apache Hudi™ vs Delta Lake vs Apache Iceberg™ - Data Lakehouse Feature Com

Thank you :)